Lab 7: Fuzzing for Input Validation Bugs

Property-based fuzz testing of pet_boarding.py with Hypothesis

Field	Details
Name	Sriviswaja M
SRN	PES1UG24AM288
Section	E
Assigned repository	SETAPESU26/L6_L7_49_pet_boarding (Lab 7)
Submission repository	github.com/sriviswaja/SE-Lab-AM288 (Lab 7 folder)

1. Objective

To use fuzz testing to find and fix input validation bugs in pet_boarding.py. Instead of hand-picking test inputs, the Hypothesis library generates hundreds of random and malformed inputs per test, shrinks any failure to the smallest input that reproduces it, and reports it.

Functions under test:

- **parse_booking(text)** parses "pet:days" (e.g. dog:5) into ("Dog", 5.0).
- **stay_average(days_csv)** averages comma-separated stay lengths (e.g. 2,5,7).
- **payable_fee(text)** parses "fee|discount", adds 18% GST and subtracts the discount.

2. Step 2: Running the Original Application

With normal, well-formed inputs the original script appears to work, which is exactly why these bugs survive ordinary manual testing:

```
(venv) (base) viswaja@Viswajas-MacBook-Air Lab 7 % python3 pet_boarding.py
==== Running parse_booking ====
Enter pet:days (e.g. dog:5): dog:5
Pet and Days: ('Dog', 5.0)

==== Running stay_average ====
Enter stay lengths in days as CSV (e.g. 2,5,7): 2,5,7
Average Stay: 4

==== Running payable_fee ====
Enter fee|discount (e.g. 1500|100): 1500|100
Payable Fee: 1670.0
```

Screenshot 1: original pet_boarding.py with valid inputs

Note that Average Stay prints **4** for 2, 5, 7. The true average is 4.67; integer division (//) silently drops the fraction.

3. Step 3: Hypothesis Fuzz Tests

test_pet_boarding.py contains seven tests in three groups:

Test	Strategy (generated input)	Property checked
test_parse_booking_no_crash test_stay_average_no_crash test_payable_fee_no_crash	st.text() | st.none()	The function never raises an exception, for any string or None.
test_parse_booking_shaped_input	f"{text}:{float}" incl. nan, inf, negatives	No crash; a returned result has a non-empty pet and finite days >= 0.

Test	Strategy (generated input)	Property checked
test_stay_average_shaped_input	lists of integers -10..60 joined by commas (incl. empty list)	No crash; a returned average is >= 0.
test_payable_fee_shaped_input	f"{float}|{float}" incl. nan, inf, huge values	No crash; a returned fee is finite and >= 0.
test_valid_examples	fixed examples	Normal inputs still give correct answers (Dog/5.0, 4.67, 1670.0).

The "no crash" tests follow the handout pattern. The "shaped" tests generate inputs that look like real bookings, which reaches deeper into each function than random text, and check that the **output makes sense**, not only that nothing crashed.

```
@given(st.text() | st.none())
def test_parse_booking_no_crash(s):
    try:
        parse_booking(s)
    except Exception as e:
        assert False, f"parse_booking crashed with: {e!r}"
```

4. Step 4: Tests Failing on the Original Code

```
test_pet_boarding.py:81: AssertionError
=================================== short test summary info ====================================
FAILED test_pet_boarding.py::test_parse_booking_no_crash - ExceptionGroup: Hypothesis found 3
distinct failures. (3 sub-exceptions)
FAILED test_pet_boarding.py::test_stay_average_no_crash - ExceptionGroup: Hypothesis found 2 d
istinct failures. (2 sub-exceptions)
FAILED test_pet_boarding.py::test_payable_fee_no_crash - ExceptionGroup: Hypothesis found 3 di
stinct failures. (3 sub-exceptions)
FAILED test_pet_boarding.py::test_parse_booking_shaped_input - ExceptionGroup: Hypothesis foun
d 2 distinct failures. (2 sub-exceptions)
FAILED test_pet_boarding.py::test_stay_average_shaped_input - ExceptionGroup: Hypothesis found
 2 distinct failures. (2 sub-exceptions)
FAILED test_pet_boarding.py::test_payable_fee_shaped_input - AssertionError: fee must be a fin
ite number
FAILED test_pet_boarding.py::test_valid_examples - AssertionError: assert 4 == 4.67
==================================== 7 failed in 1.07s =====================================
```

Screenshot 2: 7 failed. Hypothesis found 14 distinct failures across the 7 tests

Bugs found by fuzzing

Hypothesis shrinks each failure to a minimal input. The failing inputs it reported (the last rows are property violations confirmed by hand):

Function	Minimal input	Result in original code	Root cause
All three	`None`	AttributeError: 'NoneType' has no attribute 'split'	No type check
parse_booking	`"" / "dog"`	ValueError: not enough values to unpack	Assumes exactly one ':'
parse_booking	`"::"`	ValueError: too many values to unpack	Assumes exactly one ':'
parse_booking	`":"`	ValueError: could not convert string to float: ''	Empty days not checked
parse_booking	`"0:inf", "0:-1.0"`	Returns infinite or negative days (no crash)	No range check
parse_booking	`":0.0"`	Returns ("", 0.0): empty pet name accepted	Pet name not checked
stay_average	`"" / "1,,3"`	ValueError: invalid literal for int(): ''	Empty field not handled
stay_average	`"-3,1"`	Returns a negative average (no crash)	Negative stays accepted
stay_average	`"2,5,7"`	Returns 4 instead of 4.67	Integer division //
payable_fee	`"|" / ""`	ValueError on unpack / float('')	Format not validated

Function	Minimal input	Result in original code	Root cause
payable_fee	`"nan|0"`	Returns nan as the fee (no crash)	nan not rejected
payable_fee	`"1.7e308|0"`	Returns inf (overflow)	fee * 1.18 exceeds float max

Key observation: several bugs are not crashes at all. A fee of nan or an average of 4 would be silently printed to a customer. Checking properties of the output, not just "did it crash", is what exposed them.

5. Step 5: Fixing the Application

Each function was hardened with defensive checks and now returns None for invalid input instead of crashing. main() prints "Invalid input, please check the format." when it receives None.

Function	Fix
Shared helper	_to_number(text): converts with float() inside try/except and rejects nan / inf using math.isfinite().
parse_booking	Reject non-strings; require exactly one ':'; require a non-empty pet name; days must be finite and >= 0.
stay_average	Reject non-strings; catch ValueError from int() on empty or non-numeric fields; reject negative values and empty input; use true division, rounded to 2 decimals.
payable_fee	Reject non-strings; require exactly one '|'; fee and discount must be finite and >= 0; reject a non-finite total (overflow); round to 2 decimals.

Before:

```
def parse_booking(text):
    pet, days = text.split(':')
    return pet.strip().title(), float(days)
```

After:

```
def parse_booking(text):
    if not isinstance(text, str) or text.count(":") != 1:
        return None
    pet, days = text.split(":")
    pet = pet.strip()
    days = _to_number(days)       # None for '', 'abc', nan, inf
    if not pet or days is None or days < 0:
        return None
    return pet.title(), days
```

```
(venv) (base) viswaja@Viswajas-MacBook-Air Lab 7 % cp /tmp/pet_boarding_fixed.py pet_boarding.
py
[python3 pet_boarding.py                                                                      ]
==== Running parse_booking ====
Enter pet:days (e.g. dog:5): dog:5
Pet and Days: ('Dog', 5.0)

==== Running stay_average ====
Enter stay lengths in days as CSV (e.g. 2,5,7): 2,5,7
Average Stay: 4.67

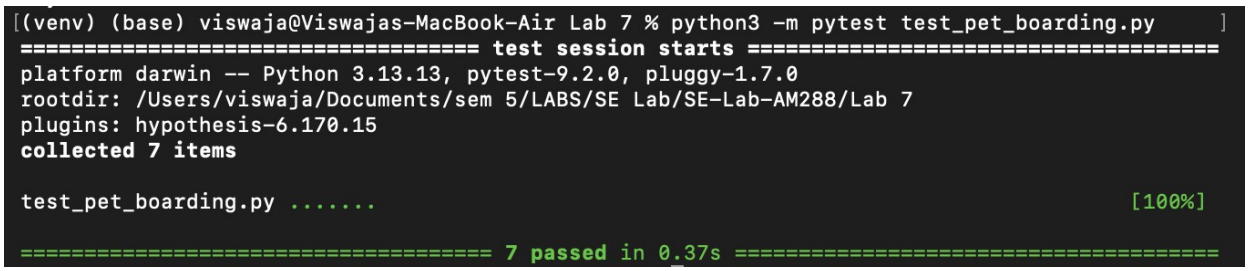
==== Running payable_fee ====
Enter fee|discount (e.g. 1500|100): 1500|100
Payable Fee: 1670.0
```

Screenshot 3: fixed pet_boarding.py. Valid inputs still work and Average Stay is now correct (4.67)

With invalid inputs, the fixed version no longer crashes:

```
Enter pet:days (e.g. dog:5): dog
Invalid input, please check the format.
Enter stay lengths in days as CSV (e.g. 2,5,7): 1,,3
Invalid input, please check the format.
Enter fee|discount (e.g. 1500|100): abc
Invalid input, please check the format.
```

6. Step 6: Tests Passing on the Fixed Code



Screenshot 4: all 7 Hypothesis tests pass against the fixed code

	Original code	Fixed code
Tests passing	0 / 7	7 / 7
Distinct failures found	14	0
Average of 2,5,7	4 (wrong)	4.67

7. Conclusion

- **Fuzzing finds what you didn't think to test.** The original script passed every normal input, yet Hypothesis found 14 distinct failures within seconds, including None, empty fields, extra separators, negative values, nan and inf.
- **Shrinking makes debugging easy.** Hypothesis reduced each failure to a minimal input such as ":" or "|", pointing straight to the faulty line.
- **Not all bugs crash.** Wrong-but-silent outputs (nan fees, negative averages, a truncated average) were caught only by asserting properties of the result.
- **Defensive programming.** Validating type, format, range and finiteness at the boundary of each function made it robust to every input the fuzzer generated, while keeping correct results for valid input.